

ANSI SQL Using MySQL — Hands-On Exercises

1. User Upcoming Events

```
SELECT e.event_id, e.title, e.start_date
FROM Events e
JOIN Registrations r ON e.event_id = r.event_id
JOIN Users u ON r.user_id = u.user_id
WHERE r.user_id = 1
      AND e.status = 'upcoming'
      AND e.city = u.city
ORDER BY e.start_date;
```

2. Top Rated Events

```
SELECT e.event_id, e.title, AVG(f.rating) AS avg_rating, COUNT(*)
AS feedback_count
FROM Events e
JOIN Feedback f ON e.event_id = f.event_id
GROUP BY e.event_id, e.title
HAVING COUNT(*) >= 10
ORDER BY avg_rating DESC;
```

3. Inactive Users

```
SELECT u.user_id, u.full_name
FROM Users u
WHERE u.user_id NOT IN (
  SELECT r.user_id FROM Registrations r
  WHERE r.registration_date >= CURDATE() - INTERVAL 90 DAY
);
```

4. Peak Session Hours

```
SELECT event_id, COUNT(*) AS session_count
FROM Sessions
WHERE TIME(start_time) >= '10:00:00' AND TIME(start_time) <
'12:00:00'
GROUP BY event_id;
```

5. Most Active Cities

```
SELECT u.city, COUNT(DISTINCT r.user_id) AS distinct_users
FROM Users u
JOIN Registrations r ON u.user_id = r.user_id
GROUP BY u.city
ORDER BY distinct_users DESC
LIMIT 5;
```

6. Event Resource Summary

```
SELECT event_id, resource_type, COUNT(*) AS total_resources
FROM Resources
GROUP BY event_id, resource_type
ORDER BY event_id;
```

7. Low Feedback Alerts

```
SELECT u.full_name, e.title, f.rating, f.comments
FROM Feedback f
JOIN Users u ON f.user_id = u.user_id
JOIN Events e ON f.event_id = e.event_id
WHERE f.rating < 3;
```

8. Sessions per Upcoming Event

```
SELECT e.event_id, e.title, COUNT(s.session_id) AS session_count
FROM Events e
LEFT JOIN Sessions s ON e.event_id = s.event_id
WHERE e.status = 'upcoming'
GROUP BY e.event_id, e.title;
```

9. Organizer Event Summary

```
SELECT organizer_id, status, COUNT(*) AS event_count
FROM Events
GROUP BY organizer_id, status
ORDER BY organizer_id;
```

10. Feedback Gap

```
SELECT DISTINCT e.event_id, e.title
FROM Events e
JOIN Registrations r ON e.event_id = r.event_id
WHERE e.event_id NOT IN (SELECT event_id FROM Feedback);
```

11. Daily New User Count

```
SELECT registration_date, COUNT(*) AS new_users
FROM Users
WHERE registration_date >= CURDATE() - INTERVAL 7 DAY
GROUP BY registration_date
ORDER BY registration_date;
```

12. Event with Maximum Sessions

```
SELECT event_id, COUNT(*) AS session_count
FROM Sessions
GROUP BY event_id
ORDER BY session_count DESC
LIMIT 1;
```

13. Average Rating per City

```
SELECT e.city, AVG(f.rating) AS avg_rating
FROM Events e
JOIN Feedback f ON e.event_id = f.event_id
GROUP BY e.city;
```

14. Most Registered Events

```
SELECT e.event_id, e.title, COUNT(r.registration_id) AS
total_registrations
FROM Events e
JOIN Registrations r ON e.event_id = r.event_id
GROUP BY e.event_id, e.title
ORDER BY total_registrations DESC
LIMIT 3;
```

15. Event Session Time Conflict

```
SELECT s1.session_id AS session1, s2.session_id AS session2,  
s1.event_id  
FROM Sessions s1  
JOIN Sessions s2 ON s1.event_id = s2.event_id AND s1.session_id <  
s2.session_id  
WHERE s1.start_time < s2.end_time AND s2.start_time < s1.end_time;
```

16. Unregistered Active Users

```
SELECT u.user_id, u.full_name  
FROM Users u  
WHERE u.registration_date >= CURDATE() - INTERVAL 30 DAY  
AND u.user_id NOT IN (SELECT user_id FROM Registrations);
```

17. Multi-Session Speakers

```
SELECT speaker_name, COUNT(*) AS session_count  
FROM Sessions  
GROUP BY speaker_name  
HAVING COUNT(*) > 1;
```

18. Resource Availability Check

```
SELECT e.event_id, e.title  
FROM Events e  
WHERE e.event_id NOT IN (SELECT event_id FROM Resources);
```

19. Completed Events with Feedback Summary

```
SELECT e.event_id, e.title,  
COUNT(DISTINCT r.registration_id) AS total_registrations,  
AVG(f.rating) AS avg_rating  
FROM Events e  
LEFT JOIN Registrations r ON e.event_id = r.event_id  
LEFT JOIN Feedback f ON e.event_id = f.event_id  
WHERE e.status = 'completed'  
GROUP BY e.event_id, e.title;
```

20. User Engagement Index

```
SELECT u.user_id, u.full_name,  
       COUNT(DISTINCT r.event_id) AS events_attended,  
       COUNT(DISTINCT f.feedback_id) AS feedbacks_given  
FROM Users u  
LEFT JOIN Registrations r ON u.user_id = r.user_id  
LEFT JOIN Feedback f ON u.user_id = f.user_id  
GROUP BY u.user_id, u.full_name;
```

21. Top Feedback Providers

```
SELECT u.user_id, u.full_name, COUNT(f.feedback_id) AS  
feedback_count  
FROM Users u  
JOIN Feedback f ON u.user_id = f.user_id  
GROUP BY u.user_id, u.full_name  
ORDER BY feedback_count DESC  
LIMIT 5;
```

22. Duplicate Registrations Check

```
SELECT user_id, event_id, COUNT(*) AS registration_count  
FROM Registrations  
GROUP BY user_id, event_id  
HAVING COUNT(*) > 1;
```

23. Registration Trends

```
SELECT DATE_FORMAT(registration_date, '%Y-%m') AS  
reg_month, COUNT(*) AS total_registrations  
FROM Registrations  
WHERE registration_date >= CURDATE() - INTERVAL 12 MONTH  
GROUP BY DATE_FORMAT(registration_date, '%Y-%m')  
ORDER BY reg_month;
```

24. Average Session Duration per Event

```
SELECT event_id, AVG(TIMESTAMPDIFF(MINUTE, start_time,  
end_time)) AS avg_duration_minutes  
FROM Sessions  
GROUP BY event_id;
```

25. Events Without Sessions

```
SELECT e.event_id, e.title  
FROM Events e  
WHERE e.event_id NOT IN (SELECT event_id FROM Sessions);
```